Screenshot Testing Strategy: BILD · Sport BILD · WELT News · WELT Edition

Goal: catch unintended visual changes on every pull request, on iOS and Android, by rendering real production UI with mocked data that is identical on every run.

In one line: native in-process snapshot tools on each platform, fed by **one shared set of JSON fixtures**, gated in CI on every PR. A small nightly pass on real simulators and emulators covers what in-process rendering cannot see.

1. Tooling

	iOS	Android
PR gate (Layer 1)	swift-snapshot-testing 1.19.7 (pinned)	Roborazzi 1.77 on Robolectric 4.17 (pinned)
Why	In-process, fast. Same authors as TCA, so mocking reuses `withDependencies`	JVM, no emulator. Works with Hilt. The same run also drives UI interaction tests
Behaviour tests	Logic and contract tests, plus XCUITest interaction tests on the pinned simulator	Logic and contract tests, plus Robolectric interaction tests (tap, type, switch tabs)
Nightly (Layer 2)	Maestro/XCUITest on a simulator, app started with `-fixture <name>`	Maestro on an emulator, `--es fixture <name>`

Rejected for the gate:

- **Paparazzi:** 2.0 alpha, no Hilt and no interactions. It's the fallback, because our test code is a thin layer over portable fixtures.
- **Google's Compose Preview Screenshot Testing:** alpha.
- **Appium/Maestro + a paid visual service:** minutes per screen instead of milliseconds, flakier, and edge states are hard to reach. It belongs to Layer 2.
- **Hosted review (Sentry Snapshots):** worth a phase-4 trial.

2. Mocked, deterministic data

- **News content changes every minute; goldens must not.** A golden is tied to a *fixture*, never to live content, so it can't go stale: fixture timestamps are relative to a pinned clock ("vor 9 Min." forever). Changing data is covered by three other mechanisms instead:
  - **contract tests** of the decoders against a schema, and nightly against real (anonymised) responses, which catch format drift
  - **fixtures refreshed from real responses** in a reviewed PR when the backend changes; that PR re-baselines the affected goldens
  - **robustness tests** that need no golden: a seeded fuzz test corrupts one feed item 300 ways, and every other item must still show
- **One fixture catalog, decoded by each platform's production decoder.** It covers typical, long-German-headline, empty, **edge-case** and **hostile-content** feeds. Hostile content means HTML entities, stray line breaks, soft hyphens, emoji, a ticker with no entries yet, a video over an hour long and a timestamp in the future.
- **The edge-case fixture holds one shared decoding contract**, asserted with the same literal on both platforms. The rules:
  - a malformed item is dropped, never the whole feed
  - a missing flag gets a safe default
  - duplicate ids keep the first entry
  - fractional-second dates parse

The two decoders had silently diverged here: one missing field blanked the iOS feed, but made a PLUS article free on Android.

- **Every source of variation is pinned:**
  - clock, `de_DE` locale, `Europe/Berlin` time zone

    - seeded randomness, with the same RNG on both platforms
    - animations off
    - local images
    - fixed device geometry, theme and text size per variant
    - ads and WebViews replaced by fixed placeholders
- **Seams.** UI and view models depend on small client values, never on the HTTP stack: `FeedClient` and `AuthClient`, plus an image provider. In the existing apps (stack *assumed from public job ads*: TCA on iOS, Hilt on Android):

Seam	iOS	Android
Data	TCA `@DependencyClient` → `withDependencies { $0.feedClient = .fixture(…) }`	Hilt `@TestInstallIn` replaces the network module
Images	Kingfisher/Nuke pipeline with a synchronous fixture loader	Coil `FakeImageLoaderEngine`
Legacy code (UIKit, singleton API client)	`URLProtocol` stub serving `/fixtures`: no refactor needed to start	OkHttp `MockWebServer` serving the same files

3. Golden images: storage, review, guards

- **Storage:** goldens live next to the tests, with Git LFS beyond about 100 MB. Names are identical on both platforms, which gives designers a side-by-side gallery.
- **Only CI records goldens:**
    1. A developer adds the `record-snapshots` label.
    2. CI re-records on the pinned image.
    3. CI **proves the new goldens before pushing them**: golden lint plus a full verify run.
    4. CI pushes them with a bot token, so the gate runs again on the new commit.
    5. Code owners (design/QA) approve the golden diff, and stale approvals are dismissed.
- **Review:** CI keeps **one sticky PR comment** up to date and attaches an HTML diff gallery (golden, this PR, changed pixels). The comment classifies each change before anyone opens an image:
    - **layout:** the size changed, so something moved
    - **visible:** pixels a person can see changed
    - **subtle:** a colour or anti-aliasing shift only, still shown so a failure never reads "0 px changed"

This is triage, not approval: every class needs a human decision. A cross-platform gallery (`make gallery`) puts iOS and Android side by side for design review, because each gate only compares a platform with itself.

- **Guards against a green test that protects nothing:**
    - **Golden lint** (CI job): fails on blank or uniform goldens, and on iOS/Android name divergence.
    - **Capture guard:** refuses to record or compare an undrawn frame.
    - **Fixture precondition tests:** "the premium teaser is premium". A fixture edit can't silently change what a golden means.
    - **Recording starts from an empty folder**, so orphaned goldens show up as deletions.

4. What is covered, and by which layer

- **Matrix (curated, not every combination):**
    - every component: light + dark
    - text-heavy and interactive screens: + largest supported text (iOS AX3, Android font scale 2.0 with real non-linear scaling)
    - error, selected and interactive states: + dark
    - feed: + smallest phone, tablet, and **full length** (not just the first screen)
    - paywall and PLUS badge: always
- **Demo coverage:** 89 goldens per platform, plus logic, contract, contrast and edge-case tests, UI interaction tests and accessibility audits on both platforms.
- **Accessibility in the gate:**
    - a WCAG contrast test runs over every text/background token pair, in light and dark
    - **automated audits on every screen:** XCUITest `performAccessibilityAudit()` on iOS and Google

ATF via Roborazzi on Android, covering contrast, touch targets, labels and traits
    - the few accepted audit exceptions are listed in code with the evidence for each (e.g. a contrast false positive on near-black text)
- **Layer 2 (nightly) covers what in-process rendering cannot:**
    - system chrome (e.g. iOS 26's Liquid Glass tab bar renders blank in-process)
    - real fonts and devices
    - **hostile device settings:** en_US 12-hour time, bold text, increased contrast. It found a "11:05 PM Uhr" bug the snapshots couldn't see.

5. CI/CD and the Scrum process

- **When it runs:**
    - **every PR**, on its merge result: golden lint, iOS (`xcode-27` runner) and Android (Linux) in parallel
    - the **merge queue**, so two PRs that are green on their own can't break main together
    - **after merge** on main
    - **nightly** on a fresh runner image (catches toolchain drift), plus the cross-platform gallery
- **Measured locally:** about 30 s (iOS) and 12 s (Android) of test time after the build. Target: under 10 minutes of wall time per PR, running only affected modules at scale.
- **Rollout of the gate:** report-only at first, then a required check.
- **Definition of Done:** a new UI state ships with a fixture and a snapshot. Story refinement asks "which states does this add?".
- **Platform upgrades** (Xcode, simulator runtime, SDK, Robolectric, the snapshot library): one PR bumps the version *and* re-baselines, reviewed as such.

6. Stability

1. **Pin the environment:** Xcode, simulator device and runtime, runner image, and all library *and test* dependencies. A transitive Espresso 3.5.1 crashed on SDK 37 until it was pinned to 3.7.0.
2. **Pin the inputs** (§2). A `Date()`, network image, animation or unseeded random value in a view is a test-design bug.
3. **Tolerance:**
    - iOS: every pixel is compared, with `perceptualPrecision 0.98`. That absorbs GPU anti-aliasing noise but fails on a 2pt shift (proven in the regression demo). Its blind spot is a colour change under ΔE≈2, which the token contrast tests and design review cover.
    - Android: zero tolerance.
    - Tolerance is never loosened globally. A per-test exception is allowed only with a comment saying why (for example, a component with a blur), and is reviewed like code.
    - No "small diff = auto-pass" rule. A 2pt shift and a dropped PLUS badge are both small; whether a change is intended is a product decision, not a pixel count.
4. **Android local runs vs Linux CI:** CI is the source of truth. Developers reproduce CI failures in the same pinned Docker image (planned `make android-test-docker`); plain local Mac runs are advisory.
5. **No automatic retries.** A flaky snapshot is quarantined within 24 hours with an owner. Target flake rate: under 1%. **Measured:** 10 consecutive full runs per platform, all green (iOS 71 tests, Android 116 tests, no retries). The first Android loop "failed" 3 of 10 only because the Mac went to sleep mid-run (wall-clock timeouts in the power log), so long local runs use `caffeinate`; a CI runner never sleeps.

7. Integration into the existing apps

- **Fixtures** live in one shared `news-fixtures` package (SPM resources plus a Gradle artifact), versioned with the API contract, validated against the OpenAPI schema, and seeded from anonymised real responses.
- **First two weeks, without blocking feature work:**
    1. Snapshot design-system components that take plain models: no network seam needed.
    2. A `URLProtocol`/`MockWebServer` stub as a stopgap.
    3. Add a client seam at one feature boundary.
    4. Run the CI job report-only.
- **Never change product UI for a test tool.** In this demo the tab bar is custom, so every pixel is testable. In

production, keep the system tab bar, snapshot the tab *content*, and cover the system chrome in Layer 2.
- **Test-support module per platform:** the deterministic wrapper, variants, the capture guard and the diff report. A new snapshot test is about 5 lines.

8. Scaling, ownership, enablement

- **Four apps share one design system.** Component goldens are shared; app modules (WELT PLUS, Sport BILD tickers) own their own screens and fixtures.
- **Cost control:** affected modules only, sharding, Git LFS, and a quarterly matrix review that drops variants that never fail.
- **Ownership:** developers write and maintain the tests. QA owns the strategy, the fixture catalog and the matrix rules. Design approves goldens.
- **Enablement:** an "add a snapshot in 5 lines" guide, pairing with each squad, a diff-review session for designers, office hours in month one, and fixture review in refinement.
- **Metrics:**
  - escaped visual bugs per release
  - PR gate time
  - flake rate
  - share of UI states with fixtures
  - contrast-test pass rate

9. Rollout (about 4 sprints)

Sprint	Step	Exit criterion
1	One app (e.g. WELT News, iOS + Android): support module, 5 design-system components, CI **report-only**	Green for 5 days, no flakes
2	Shared fixtures, feed and paywall states, diff gallery and sticky comment, CODEOWNERS; pairing with each squad	Every squad merged at least one snapshot test
3	Gate **required** and in the Definition of Done; CI-only recording; Layer 2 nightly	Under 10 min, flake rate under 1%
4+	Other three apps; accessibility snapshots; trial a hosted review tool	Visual bugs caught by the gate, not in release testing

---

Appendix A: evidence from building the demo

Product bugs caught before any deliberate regression:

- the banner timestamp was grey on red
- the kicker was truncated, and "EILMEL-DUNG" hyphenated, at large text sizes
- Android didn't hyphenate German headlines while iOS did (found by comparing the same fixture across platforms)
- a 60-letter compound word breaks mid-word on both platforms. This is a **content** finding: the fix is soft hyphens from the CMS
- an e-mail placeholder rendered as a Markdown link
- "11:05 PM Uhr" on an en_US device (found by Layer 2)

Found by the edge-case and accessibility round:

- a video over one hour showed "62:03"
- Android used the device locale for durations, so an Arabic phone would show non-Latin digits
- a live ticker without entries was dropped from the feed
- raw `&` and line breaks from the CMS would reach the screen
- a fixture image without `.png` loaded on iOS but not on Android (the image contract now checks exact file names)
- **hidden tabs were still reachable by VoiceOver** (found by the iOS audit); the sign-out button failed contrast
- after a failed refresh, Android inserted the offline notice just above the visible list, so readers never saw it (found by the pull-to-refresh interaction test)
- Google's official Compose accessibility hook silently checks nothing under Robolectric: a deliberately

broken button passed. A guard that can't fail protects nothing, so every new check was first shown to fail.

Tooling incidents, each now guarded against:

- 44 of 62 Android goldens blank but green (capture guard and golden lint)
- a transitive Espresso crash on SDK 37 (pinned test dependencies)
- the Liquid Glass tab bar can't be captured in-process (custom bar in the demo; Layer 2 in production)
- tall components clipped by the harness
- stale diff files shown to reviewers (cleared before every run)
- a stale app build launched by the run script (project-local build, always rebuilt)
- the pinned simulator runtime vanished mid-session (re-pin plus re-baseline in one commit)

Five role-based reviews, by a product manager, a senior developer, a senior QA, an accessibility/design reviewer and a hiring-panel reviewer, found:

- the missing paywall/entitlement states
- decoding differences between the platforms
- Android losing state on tab switches and rotation
- Android dark-mode buttons that were purple on red (about 2.3:1) in *approved* goldens
- undersized iOS game boards
- low-contrast secondary text
- CI record-mode gaps

All are fixed and covered by tests. **Deliberate regression** (PLUS badge dropped, banner −2pt): the **same 29 of 89** snapshots fail on both platforms, including a real WELTplus teaser silently losing its paywall badge. Every unaffected screen (games, account, paywall) stays green.

Appendix B: devil's advocate

Objection	Answer
"Reviewers will approve diffs without looking."	Small focused snapshots, code-owner approval, stale approvals dismissed, and the diff is in the PR
"In-process rendering isn't the real app."	True. That's Layer 2's job (system chrome, devices, hostile settings)
"Every OS or Xcode upgrade changes every golden."	Yes, as a planned re-baseline PR per upgrade
"Too many combinations."	Curated matrix rules, affected-modules-only runs, quarterly pruning
"Roborazzi is largely one maintainer."	Exact pinning, and thin test code over portable fixtures: switching to Paparazzi is cheap
"QA should own the tests."	Developers own them. QA owns the system around them
"Can AI approve diffs?"	AI triages (summarises diffs, generates edge-case fixtures). A human approves goldens